

maquette-gltf

Render glTF 2.0 assets in Typst

github.com/bernsteining/maquette · bernsteining.github.io/maquette



Version 0.1.0 · August 26, 2026

CONTENTS

Contents

Introduction	3
Where to find sample glTF assets	4
Quickstart	4
Shared rendering config	6
Image-Based Lighting	7
Procedural IBL	7
HDR environment	7
IBL rotation	7
Animations	8
Material variants	9
Diffuse transmission	9
Compression + quantization	10
EXT_meshopt_compression	10
KHR_draco_mesh_compression	10
KHR_mesh_quantization	10
Texture formats	11
EXT_texture_webp	11
Unsupported formats	11
Metadata	12
What's supported vs. not	13

Introduction

maquette-gltf extends **maquette** with glTF 2.0 support: `.glb` or `.gltf` in, rendered frame out. The plugin adds a PBR pipeline (Cook-Torrance GGX + Karis split-sum IBL + WBOIT translucency) on top of **maquette**'s shared rasterizer, plus glTF-specific machinery for authored cameras/lights/materials/animations and the ecosystem's compression + texture extensions.

This document only covers what **maquette-gltf** **adds**. The shared rendering knobs — camera framing, background, shadows, ground plane, SSAO/FXAA/SSAA, tone mapping — are already documented in **maquette**'s manual and behave identically here. `render-gltf` accepts the same option dicts.

Where to find sample glTF assets

The examples below use canonical glTF sample models (Damaged Helmet, Fox, CesiumMan, ToyCar). Any `.glb` or `.gltf` works — the plugin handles compression extensions (Draco, meshopt, KTX2, WebP), quantization, and animations transparently.

- KhronosGroup/glTF-Sample-Assets — the canonical sample set every glTF viewer uses. Every extension has a dedicated test asset here.
- Poly Haven Models — CC0, glTF-native. High-quality PBR assets with authored materials.
- Sketchfab (downloadable filter) — largest general library; downloads default to glTF.

For the HDR environment maps that feed the IBL section:

- Poly Haven HDRIs — CC0 8K/16K equirectangular HDRs. Bundle with your `.typ` and pass the bytes to `ibl: (hdr: ...)`.

Quickstart

Two input shapes, both handled transparently.

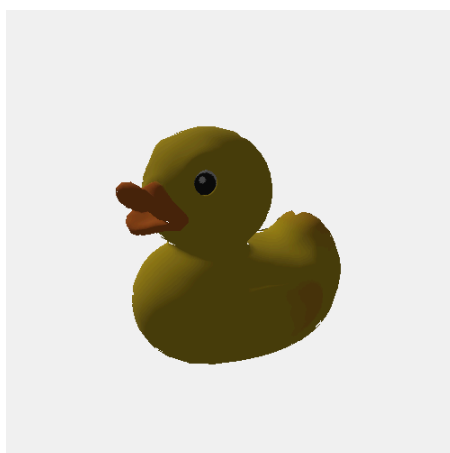
.glb or fully-embedded .gltf — pass the bytes directly. This is the common case (Damaged Helmet, boombox, most Sketchfab downloads).

```
#import "@preview/maquette-glTF:0.1.0": render-glTF
#render-glTF(read("helmet.glb", encoding: none))
```



Split .gltf — the `.gltf` JSON references external `.bin` + textures by relative URI. Pass the **path string** plus an inline `read: lambda`; the wrapper walks the JSON, discovers every URI, reads each one through your `lambda`, packs them into a sidecar bundle for the plugin. You write zero filenames.

```
#render-glTF("Duck.glTF", read: p => read(p, encoding: none))
```



The `read: lambda` has to be an **inline lambda**, not a bare read reference. Typst resolves `read()` paths against the source file the call is textually in — a bare reference stays bound to the package’s own path context. Wrapping in `p => read(p, ...)` gives the wrapper a filesystem handle scoped to your `.typ`.

Shared rendering config

Camera framing (camera / center / up / fov / azimuth / elevation), background, shadows, ground, ssao, antialias, tone_mapping — all inherited from `maquette` and behave the same on `render-gltf`. Don't repeat the tour here; refer to that manual.

One glTF-only knob to note: **camera_auto_use: false**. If the loaded asset ships an authored camera, `render-gltf` uses it by default (glTF viewer convention). Set this to `false` to force your Cartesian/spherical arguments to win instead.

```
#render-gltf(helmet, read: R,  
  camera_auto_use: false,  
  camera: (0, 0, 3), up: (0, 1, 0), fov: 60,  
)
```



Image-Based Lighting

The dominant visual lever for PBR: an environment map lights the whole scene. Enabling ibl without an hdr bytes payload falls back to a procedural hemispheric sky.

Procedural IBL

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.2),  
)
```



HDR environment

Pass Radiance .hdr bytes as ibl.hdr. The prefilter builds a diffuse irradiance map + a specular mip chain up-front (once per HDR, cached).

```
#render-gltf(helmet, read: R,  
  ibl: (intensity: 1.0, hdr: read("studio.hdr", encoding: none)),  
)
```

IBL rotation

The rotation field spins the environment around the up axis (radians). Useful for aligning studio HDRs to your camera composition.

```
#render-gltf(helmet, read: R,  
  camera: (2.5, 1.5, 2.5), up: (0, 1, 0), fov: 40,  
  background: "#181820",  
  ibl: (intensity: 1.2, rotation: 1.2),  
)
```



Animations

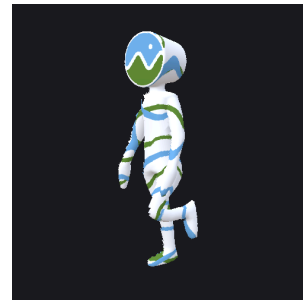
Assets with animation channels get replayed at time seconds. `get-gltf-info` returns `max_animation_time` so you can build a scrub slider bounded to the actual clip length.

```
#render-gltf(fox,
  camera: (120, 90, 180), center: (0, 40, 0), up: (0, 1, 0),
  fov: 40,
  background: "#1a1a22",
  ibl: (intensity: 1.3),
  time: 0.5,
  width: 45%,
)
```



When the asset ships multiple clips (idle / walk / run / ...), pick one with `animation_index`. None (the default) plays every clip stacked — last-write-wins per node channel — which is almost never what you want for multi-clip assets.

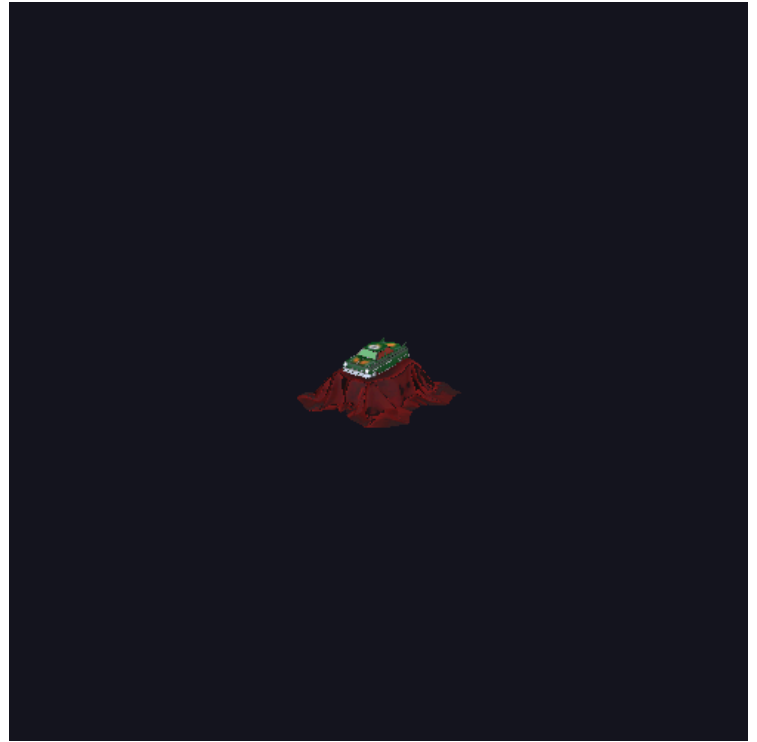
```
#render-gltf(cesiumman,
  camera: (2.5, 1, 2.5), center: (0, 0.9, 0), up: (0, 1, 0),
  fov: 30,
  background: "#1a1a22",
  ibl: (intensity: 1.3),
  animation_index: 0,
  time: 0.3,
  width: 40%,
)
```



Material variants

KHR_materials_variants lets an asset ship multiple material sets. Pick one with `material_variant` (0-indexed).

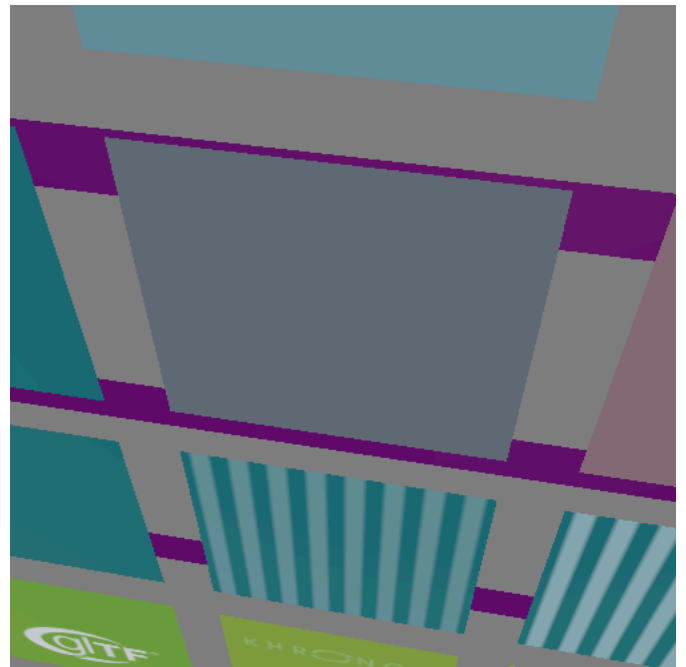
```
#render-gltf(toycar,  
  camera: (0.4, 0.3, 0.55), center: (0, 0.05, 0), up: (0, 1, 0),  
  fov: 30,  
  background: "#181820",  
  ibl: (intensity: 1.4),  
  material_variant: 0,  
)
```



Diffuse transmission

KHR_materials_diffuse_transmission — matte back-lit lambertian. Common on cloth, leaves, paper. The example asset is a factor $\times$ color grid; the bottom rows show the green DT color activating as the factor increases.

```
#render-gltf(dt-test,  
  camera: (1.5, 0.5, 1.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  background: "#222",  
  ibl: (intensity: 1.0),  
  width: 90%,  
)
```



Compression + quantization

The plugin decodes three common wire formats up-front, so downstream traversal never has to care.

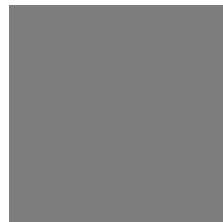
EXT_meshopt_compression

Emitted by gltfpack (Meshopt's own tool). Interleaved buffer decompression, transparent.

KHR_draco_mesh_compression

Google Draco. Decoded via draco-oxide (pure Rust). Up to 20× geometry compression on typical assets, no size penalty at render time.

```
#render-gltf(box-draco, read: R,  
  camera: (2, 1, 2), center: (0, 0, 0), up: (0, 1, 0), fov: 40,  
  background: "#333",  
  width: 30%,  
)
```



KHR_mesh_quantization

gltfpack also emits quantized POSITION/NORMAL/TANGENT (i8/u8/i16/u16) paired with KHR_texture_transform for UV dequantization. Renders pixel-identically to the non-quantized asset — verified on the Duck below.

```
#render-gltf(duck-quant, read: R,  
  camera: (2.5, 1.5, 2.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  width: 40%,  
)
```



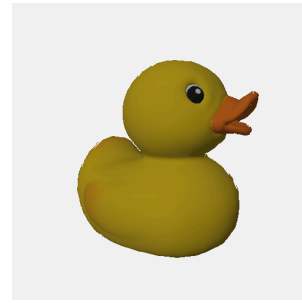
Texture formats

Beyond the base PNG + JPEG:

EXT_texture_webp

image-webp (pure Rust) handles both VP8 and VP8L variants. Second-most-common texture format in production glTF (Shopify AR, IKEA, Blender’s default export in 3.4+).

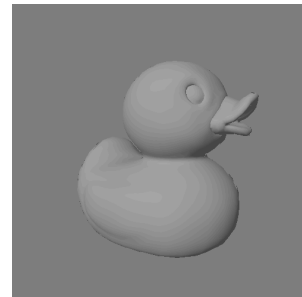
```
#render-gltf(duck-webp, read: R,  
  camera: (2.5, 1.5, 2.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  width: 40%,  
)
```



Unsupported formats

KHR_texture_basisu (KTX2 / Basis Universal) and EXT_texture_avif aren’t decoded — they’d pull in 1 MB of decoder deps for formats we ship without today. Assets using them render with a **placeholder white texture** per material (geometry stays intact) rather than crashing. Below: the same Duck with KTX2 textures, silhouette-shaded because the base color reads as white:

```
#render-gltf(duck-ktx, read: R,  
  camera: (2.5, 1.5, 2.5), center: (0, 0, 0), up: (0, 1, 0),  
  fov: 40,  
  background: "#333",  
  width: 40%,  
)
```



Workaround: pre-transcode via [gltf-transform](https://gltf-transform.dev) to WebP or PNG.

Metadata

`get-gltf-info` returns a dict — bounding box, triangle count, animation length. Useful for driving auto-framing or a scrub slider without touching the render path.

```
#let info = get-gltf-info(helmet, read: R)
#{
  raw(block: true, lang: "yaml",
    "triangles: " + str(info.triangles) + "\n" +
    "bbox_min: [" + info.bbox_min.map(x => str(calc.round(x, digits: 2))).join(", ")
+ "]" + "\n" +
    "bbox_max: [" + info.bbox_max.map(x => str(calc.round(x, digits: 2))).join(", ")
+ "]" + "\n" +
    "center: [" + info.center.map(x => str(calc.round(x, digits: 2))).join(", ") +
+ "]" + "\n" +
    "radius: " + str(calc.round(info.radius, digits: 2)) + "\n" +
    "max_animation_time: " + str(info.max_animation_time)
  )
}
```

```
triangles: 15452
bbox_min:  [-0.95, -0.9, -1.19]
bbox_max:  [0.94, 0.9, 0.81]
center:    [0, 0, -0.19]
radius:    1.64
max_animation_time: 0
```

What's supported vs. not

Core spec: glTF 2.0 JSON + GLB (single-file and split with external .bin/textures), meshes (POINTS/LINES/TRIANGLES), skinning (JOINTS_0 + JOINTS_1, WEIGHTS_0 + WEIGHTS_1, up to 8 influences per vertex), morph targets, animations (TRS + morph weights, all three interpolation modes), multiple UV sets (TEXCOORD_0/1/2), vertex colors, cameras (perspective + orthographic), multiple scenes, multiple animations, textures with samplers, sparse accessors, KHR_mesh_quantization.

Rendering: Cook-Torrance GGX PBR, IBL from HDR (Radiance / RGBE), shadow maps (PCF + PCSS), alpha modes (OPAQUE / MASK / BLEND via WBOIT), double-sided normal flip, SSAO, FXAA, SSAA $\times 2/\times 4$.

Extensions: KHR_lights_punctual, KHR_materials_unlit, KHR_materials_transmission, KHR_materials_ior, KHR_materials_specular, KHR_materials_emissive_strength, KHR_materials_volume, KHR_materials_clearcoat, KHR_materials_sheen, KHR_materials_iridescence, KHR_materials_anisotropy, KHR_materials_dispersion, KHR_materials_diffuse_transmission, KHR_texture_transform, KHR_materials_pbrSpecularGlossiness, KHR_materials_variants, EXT_texture_webp, EXT_meshopt_compression, KHR_draco_mesh_compression.

Not supported (assets render with graceful fallback — placeholder white texture or silent skip):

- KHR_texture_basisu (KTX2). Biggest gap — dominant production texture format.
- EXT_texture_avif. Rare in the wild.
- KHR_animation_pointer. Animations targeting material / light / camera properties via JSON pointer are ignored.
- TEXCOORD_N for $N \geq 3$ (collapses to slot 2).
- Sparse accessors combined with quantization.

See [crates/maquette-gltf/README.md](#) for the full compliance matrix and per-extension notes.